

Computational Complexity, Floating Point & Convergence

How fast, how accurate, how close?

Sharareh Sayyad

Washington State University

January 30, 2026

Outline

Computational complexity and Big- O

Floating point numbers

Convergence of sequences

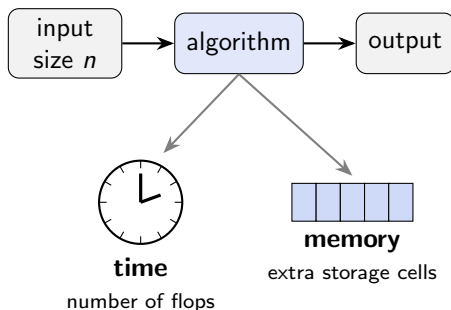
Summary and practice

What is computational complexity?

The **computational complexity** of an algorithm is the amount of **resources** it needs, written as a function of the **input size** n .

Two resources matter most:

- ▶ **Time complexity:** how many **flops** it performs. A *flop* is one floating point operation: $+$, $-$, $\times$, $\div$.
- ▶ **Space complexity:** how much *extra* memory it needs, beyond the input itself.



Counting flops: the function $f(n)$

Count the flops of a function that computes the mean of a list of length n :

```
def mean(x):  
    s = 0.0          # 0 flops  
    for a in x:      # n times:  
        s = s + a    # 1 flop  
    return s/len(x)  # 1 flop
```

$$f(n) = \underbrace{n}_{\text{additions}} + \underbrace{1}_{\text{division}} = n + 1$$

- ▶ **Time:** $f(n) = n + 1$ flops.
- ▶ **Memory:** one extra variable s , whatever n is.

Assignments, comparisons and the loop counter are not flops.

n	flops $f(n)$
10	11
100	101
1 000	1 001
1 000 000	1 000 001

Observation

For large n the “+1” hardly matters.
What matters is *how* $f(n)$ grows:
double n , double the flops.

Big-O: an upper bound on growth

Definition

$f(n) = O(g(n))$ if there are constants $c > 0$ and n_0 with

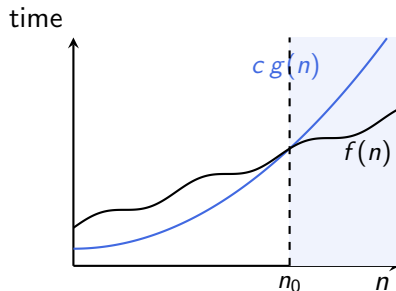
$$f(n) \leq c g(n) \quad \text{for all } n \geq n_0.$$

- ▶ “ f grows no faster than g ”.
- ▶ Drop constants and smaller terms:
 $3n^2 + 10n + 50 = O(n^2)$.

Two relatives, briefly:

$\Omega(g)$ lower bound: $f(n) \geq c g(n)$

$\Theta(g)$ tight: both $O(g)$ and $\Omega(g)$



After n_0 , f stays below $c g$.

Big- O in practice: reading loops

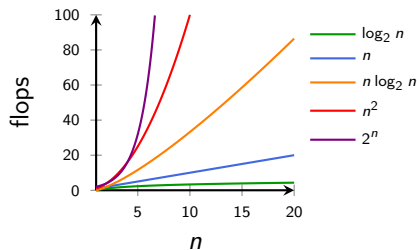
Count how many times the loop body runs, then keep the biggest term and drop constants.

loop pattern	rounds	cost
fixed number (e.g. 10)	10	$O(1)$
$i += c$ up to n	n/c	$O(n)$
$i *= 2$ up to n	$\log_2 n$	$O(\log n)$
two nested loops to n	$n \cdot n$	$O(n^2)$
loop, then another loop	$n + n$	$O(n)$

Two rules

Nested loops **multiply**; loops **one after another add**.

Example: a double loop, then a single loop:
 $n^2 + n = O(n^2)$.



The doubling test

Double n , watch the flop count:

$\times 2 \Rightarrow O(n)$, $\times 4 \Rightarrow O(n^2)$, $\times 8 \Rightarrow O(n^3)$,
 $+1 \Rightarrow O(\log n)$.

Big- O also describes **errors**: later we write error
 $= O(1/n)$ or $O(0.67^n)$.

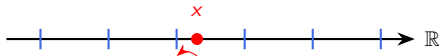
A computer cannot store every real number

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

- ▶ A number is stored in a **fixed number of bits** (64 for a Python float).
- ▶ Only finitely many numbers fit, so most real numbers are **rounded** to the nearest one that does.
- ▶ 0.1 has no exact binary form, just as $1/3 = 0.333\dots$ has no exact decimal form.

0.1 in binary repeats forever:

$$0.1 = 0.00011001100110011\dots_2$$



x is rounded to the
nearest machine number

Scientific notation

Write a number as

$$x = \pm m \times 10^e, \quad 1 \leq m < 10$$

- ▶ m , the **mantissa**, holds the digits;
- ▶ e , the **exponent**, is an integer giving the size.

The digits of m are the **significant digits**.

Why “floating point”?

$$\begin{array}{c} 0.00042 \\ \downarrow \text{point moves} \\ \downarrow 4 \text{ places} \\ 4.2 \times 10^{-4} \end{array}$$

The point **floats** to just after the first nonzero digit; the exponent remembers where it was.

number	299 792 458	0.00042	−1500
scientific	2.99792458×10^8	4.2×10^{-4}	-1.5×10^3
Python	2.99792458e8	4.2e-4	-1.5e3
significant digits	9	2	2

Huge and tiny numbers use the *same* number of digits: precision is **relative** to the size.

A toy computer with 3 significant digits

Store every number as $\pm d.dd \times 10^e$, with $-9 \leq e \leq 9$.

- ▶ **Rounding:** $1/3 \rightarrow 3.33 \times 10^{-1}$,
 $2/3 \rightarrow 6.67 \times 10^{-1}$.
- ▶ **Gaps grow:** between 1 and 10 the gap is 0.01, between 10 and 100 it is 0.1, between 100 and 1000 it is 1.
- ▶ **Relative error** of rounding: at most $\frac{1}{2} \times 10^{-2}$ (0.5%), for every size.
- ▶ **Range:** largest 9.99×10^9 , smallest positive 1.00×10^{-9} ; beyond that, overflow and underflow.

The same idea on a real computer:

	toy	double
base	10	2
mantissa	3 digits	53 bits
decimal digits	3	≈ 16
rel. error	5×10^{-3}	1.1×10^{-16}
exponent	$[-9, 9]$	$[-1022, 1023]$

Key idea

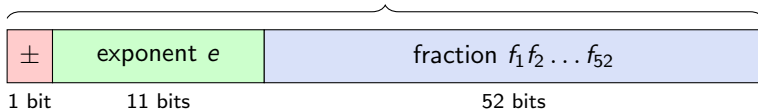
Floating point = scientific notation with a **fixed number of digits**. Gaps, ε and rounding all come from this, in base 2.

The floating point format

Scientific notation in base 2, with a fixed number of bits:

$$x = \pm (1.f_1 f_2 \dots f_{52})_2 \times 2^e$$

64 bits (IEEE 754 double precision)



Fraction (52 bits)

decides the **precision**: about 16 significant decimal digits.

Exponent (11 bits)

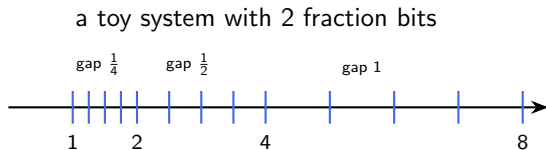
decides the **range**: from about 10^{-308} to 10^{308} .

The number line has gaps, and they grow

- ▶ Between two powers of 2 the machine numbers are **evenly spaced**.
- ▶ Each time we pass a power of 2, the spacing **doubles**.
- ▶ So the *relative* spacing stays about the same everywhere.

x	gap to the next float
1	2.2×10^{-16}
10^8	1.5×10^{-8}
10^{16}	2

`(np.spacing(x))` so `1e16 + 1 == 1e16` is True!



Machine epsilon and rounding error

Machine epsilon

$\varepsilon = 2^{-52} \approx 2.2 \times 10^{-16}$, the gap between 1 and the next float.

`(np.finfo(float).eps)`

Rounding

Storing x gives $\text{fl}(x) = x(1 + \delta)$ with $|\delta| \leq \varepsilon/2$.

Every flop ($+$, $-$, $\times$, $/$) is computed exactly, then rounded: a **relative** error of at most $\varepsilon/2$ each time.

Rule of thumb

A double carries about **16 significant digits**. Asking for more accuracy than $\approx 10^{-16}$ (relative) is asking for the impossible.

Finding ε yourself:

```
eps ← 1
while 1 + eps/2 > 1:
    eps ← eps/2
```

(Problem 2 in the notebook)

Range: overflow, underflow, inf and nan

situation	example	result
largest float	<code>np.finfo(float).max</code>	$\approx 1.8 \times 10^{308}$
overflow	<code>1e308 * 10</code>	<code>inf</code>
smallest normal float	<code>np.finfo(float).tiny</code>	$\approx 2.2 \times 10^{-308}$
underflow	<code>1e-308 / 1e100</code>	<code>0.0</code>
undefined	<code>inf - inf, 0 * inf</code>	<code>nan</code> (not a number)

Watch out

nan spreads: any calculation with a nan gives nan, and `nan == nan` is False. Use `np.isnan(x)` to test for it.

Pitfall 1: comparing and adding floats

Never test floats with ==

```
x = 0.1 + 0.2
x == 0.3                # False
abs(x - 0.3) < 1e-12   # True
np.isclose(x, 0.3)     # True
```

Compare with a **tolerance**.

Small plus large loses the small

```
1e16 + 1 - 1e16  # 0.0
1e16 - 1e16 + 1  # 1.0
```

The order of operations matters: floating point addition is **not associative**.

Summing many small numbers into a big total slowly loses digits.

Pitfall 2: catastrophic cancellation

Subtracting two **nearly equal** numbers wipes out the leading digits; only the rounding errors are left.

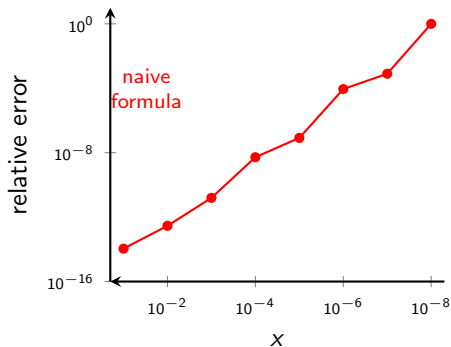
Example: $f(x) = \frac{1 - \cos x}{x^2} \rightarrow \frac{1}{2}$ as $x \rightarrow 0$.

For small x , $\cos x \approx 1$, so $1 - \cos x$ cancels.

Fix: rewrite the formula

$1 - \cos x = 2 \sin^2(x/2)$ has no subtraction:

$f(x) = \frac{2 \sin^2(x/2)}{x^2}$ is accurate for every x .



The smaller x , the bigger the error: at $x = 10^{-8}$ the naive formula returns 0 instead of 0.5 (100% error).

What does “a sequence converges” mean?

A sequence $x_1, x_2, x_3, \dots$ **converges** to L if the error

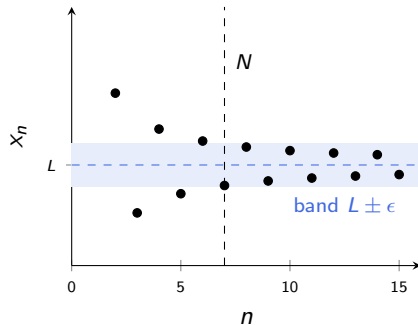
$$e_n = |x_n - L|$$

goes to 0 as $n \rightarrow \infty$.

Precisely

For every tolerance $\epsilon > 0$ there is an N such that $|x_n - L| < \epsilon$ for all $n \geq N$.

“Pick any band around L ; eventually the sequence stays inside it.”



$x_n = 1 + (-1)^n/n$: from N on, every point stays in the band.

Three example sequences

n	$(1 + \frac{1}{n})^n$	$x_{n+1} = \cos x_n$	$x_{n+1} = \frac{1}{2} \left(x_n + \frac{2}{x_n} \right)$
1	2.000000	1.000000	1.000000000000
2	2.250000	0.540302	1.500000000000
3	2.370370	0.857553	1.416666666667
4	2.441406	0.654290	1.414215686275
5	2.488320	0.793480	1.414213562375
limit	$e = 2.718281\dots$	$0.739085\dots$	$\sqrt{2} = 1.414213562373\dots$

a formula in n
slow

repeat a function (fixed point)
steady

Newton's method for $x^2 = 2$
very fast

Question

All three converge. How do we measure *how fast*?

Rates of convergence

Compare each error with the previous one:

name	rule	example
sublinear	$e_n \approx C/n$	$(1 + \frac{1}{n})^n$: $e_n = O(1/n)$
linear	$e_{n+1} \approx \rho e_n, \quad 0 < \rho < 1$	cos iteration, $\rho \approx 0.67$
quadratic	$e_{n+1} \approx C e_n^2$	Newton's method: digits double

Linear

Error shrinks by the same factor ρ every iteration: $e_n = O(\rho^n)$. With $\rho = 0.67$ we gain one digit about every 6 iterations.

Quadratic

$e_n \approx 10^{-3} \Rightarrow e_{n+1} \approx 10^{-6} \Rightarrow 10^{-12}$. Newton for $\sqrt{2}$: errors 2×10^{-3} , 2×10^{-6} , 2×10^{-12} .

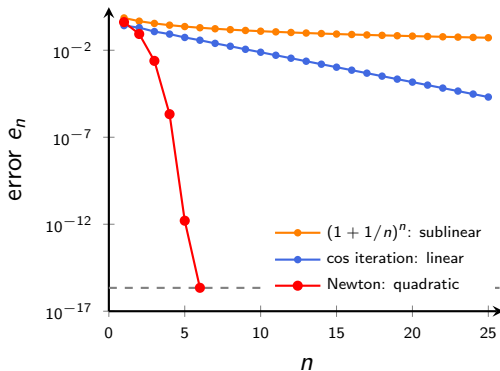
Seeing the rate: plot the error on a log scale

Plot e_n with `plt.semilogy`:

- ▶ **linear**: a straight line (slope $\log \rho$);
- ▶ **quadratic**: bends down, steeper and steeper;
- ▶ **sublinear**: flattens out.

The floor

Newton reaches $\approx 10^{-16}$ in 5 iterations and cannot go lower: that is machine epsilon.



Estimating the order from data

If $e_{n+1} \approx C e_n^p$, then p is the **order** of convergence ($p = 1$ linear, $p = 2$ quadratic). From three consecutive errors:

$$p \approx \frac{\log(e_{n+1}/e_n)}{\log(e_n/e_{n-1})}$$

Newton for $\sqrt{2}$

errors:

8.6×10^{-2} , 2.5×10^{-3} , 2.1×10^{-6} , 1.6×10^{-12}

estimates of p : 1.98, 2.00 $\Rightarrow$ quadratic.

Linear: estimate ρ

$\rho \approx e_{n+1}/e_n$.

cos iteration: $\rho \approx 0.67 = |\sin(0.739)|$.

Use errors that are well above 10^{-16} : near the floor the estimates become noise.

When floating point meets convergence

In exact arithmetic $(1 + \frac{1}{n})^n \rightarrow e$. On a computer:

n	computed $(1 + \frac{1}{n})^n$
10^6	2.7182804690
10^8	2.7182817983
10^{12}	2.7185234960
10^{15}	3.0350352065
10^{16}	1.0000000000
e	2.7182818284

At $n = 10^{16}$, $1 + 10^{-16}$ rounds to exactly 1.

Lesson

" $n \rightarrow \infty$ " is a mathematical idea. On a computer, rounding error eventually wins.

Stopping an iteration

Stop when the change per iteration gets small:

$$|x_{n+1} - x_n| < \text{tol}$$

and choose tol well above $10^{-16} |x_n|$. Always add a maximum number of iterations.

Summary

Complexity

- ▶ time = flops, space = memory
- ▶ O upper bound; Ω lower, Θ tight
- ▶ drop constants and small terms

Floating point

- ▶ scientific notation in base 2
- ▶ 64 bits, about 16 digits
- ▶ $\varepsilon \approx 2.2 \times 10^{-16}$
- ▶ compare with a tolerance
- ▶ avoid subtracting nearly equal numbers

Convergence

- ▶ $e_n = |x_n - L| \rightarrow 0$
- ▶ sublinear, linear, quadratic
- ▶ semilogy shows the rate
- ▶ floor at $\approx 10^{-16}$

Big- O connects them: cost = $O(n^2)$, error = $O(1/n)$, error = $O(\rho^n)$.